

Piotr Bonar
Artificial Intelligence
Problems nr 2

Exercise 1.

Representing the problem

- State space: all valid positions in the room
- Initial state: robots position on beginning
- Goal state: position, where robot I supposed to do the action
- Action space: all the possible moves (up, down, right, left)
- Cost function: step counter, every step has a cost of 1

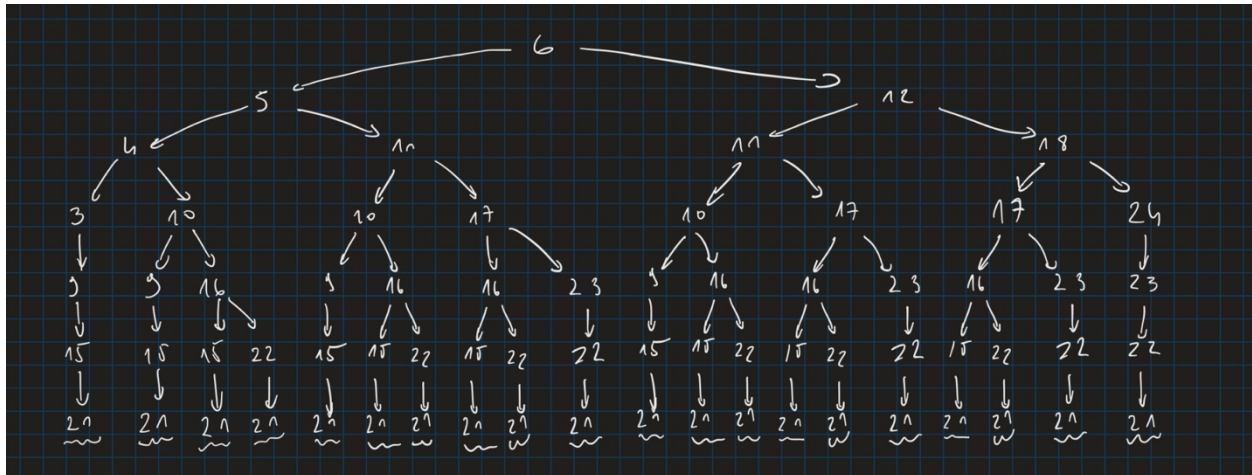
Branches, nodes and branching factors

- Nodes: position where robot can be
- Branches: Valid moves from one node to another
- Branching factor: number of possible moves from one node to other ones

The best Algorithms

- BFS: guarantees the shortest path in terms of steps
- DFS: It could have sense if we include cycles detection (makes most sense if we don't plan to first find the root and then start moving, just start moving, because this way we won't have to go back that many times)
- Uniform Cost Search: also guarantees the shortest path

The Tree



Exercise 2.

a)

BFS:

Iteration 1 – [[1]]

Iteration 2 – [[2, 1], [3, 1]]

Iteration 3 – [[4, 2, 1], [5, 2, 1], [6, 3, 1], [7, 3, 1]]

Iteration 4 – [[8, 4, 2, 1], [9, 4, 2, 1], [10, 5, 2, 1], [11, 5, 2, 1], [6, 3, 1], [7, 3, 1]]

Output: [11, 5, 2, 1]

Cost: 18

DFS:

Iteration 1 – [[1]]

Iteration 2 – [[2, 1]]

Iteration 3 – [[4, 2, 1]]

Iteration 4 – [[8, 4, 2, 1]] (no possible moves, come back)

Iteration 5 – [[9, 4, 2, 1]]

Iteration 6 – [[13, 9, 4, 2, 1]]

Iteration 7 – [[19, 13, 9, 4, 2, 1]]

Output: [19, 13, 9, 4, 2, 1]

Cost: 37

UCS:

Iteration 1 – [(0, [1])]

Iteration 2 – [(3, [1,2]), (4, [1,3])]

Iteration 3 – [(4, [1,3]), (13, [1,2,5]), (23, [1,2,4])]

Iteration 4 – [(7, [1,3,6]), (13, [1,2,5]), (23, [1,2,4])]

Iteration 5 – [(9, [1,3,6,12]), (13, [1,2,5]), (23, [1,2,4])]

Iteration 6 – [(10, [1,3,6,12,18]), (13, [1,2,5]), (16, [1,3,6,12,17]), (23, [1,2,4])]

Iteration 7 – [(12, [1,3,6,12,18,22]), (13, [1,2,5]), (16, [1,3,6,12,17]), (23, [1,2,4])]

Output: [1,3,6,12,18,22]

Cost: 12

b)

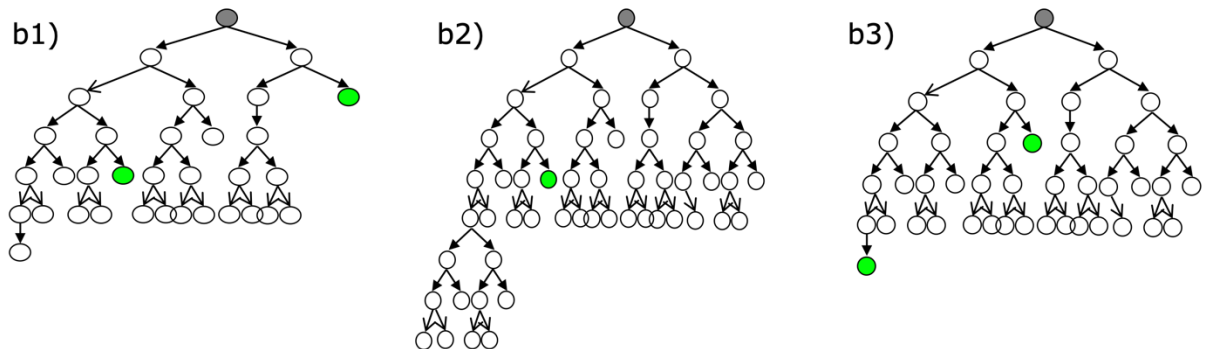
Q. 1

What algorithms guarantee that they will always come up with a solution if it exists?

BFS (if graph is not infinite), UCS (if all step costs positive), DFS (if no infinite loops)

Q. 2

2. For situations that generate the following trees, where the green nodes represent target nodes, which algorithm will arrive faster at a solution in each case? And which will take longer ? we can count the time in terms of open nodes.



I do not take UCS into account because the costs aren't shown and if all the jumps are worth the same it would work like BFS

b1)

the fastest: BFS (~ 7 nodes)

the slowest: DFS (~ 17 nodes)

b2)

the fastest: BFS (~ 18 nodes)

the slowest: DFS (~ 23 nodes if always left one is first)

b3)

the fastest: DFS (~ 7 nodes)

the slowest: BFS (~ 11 nodes)